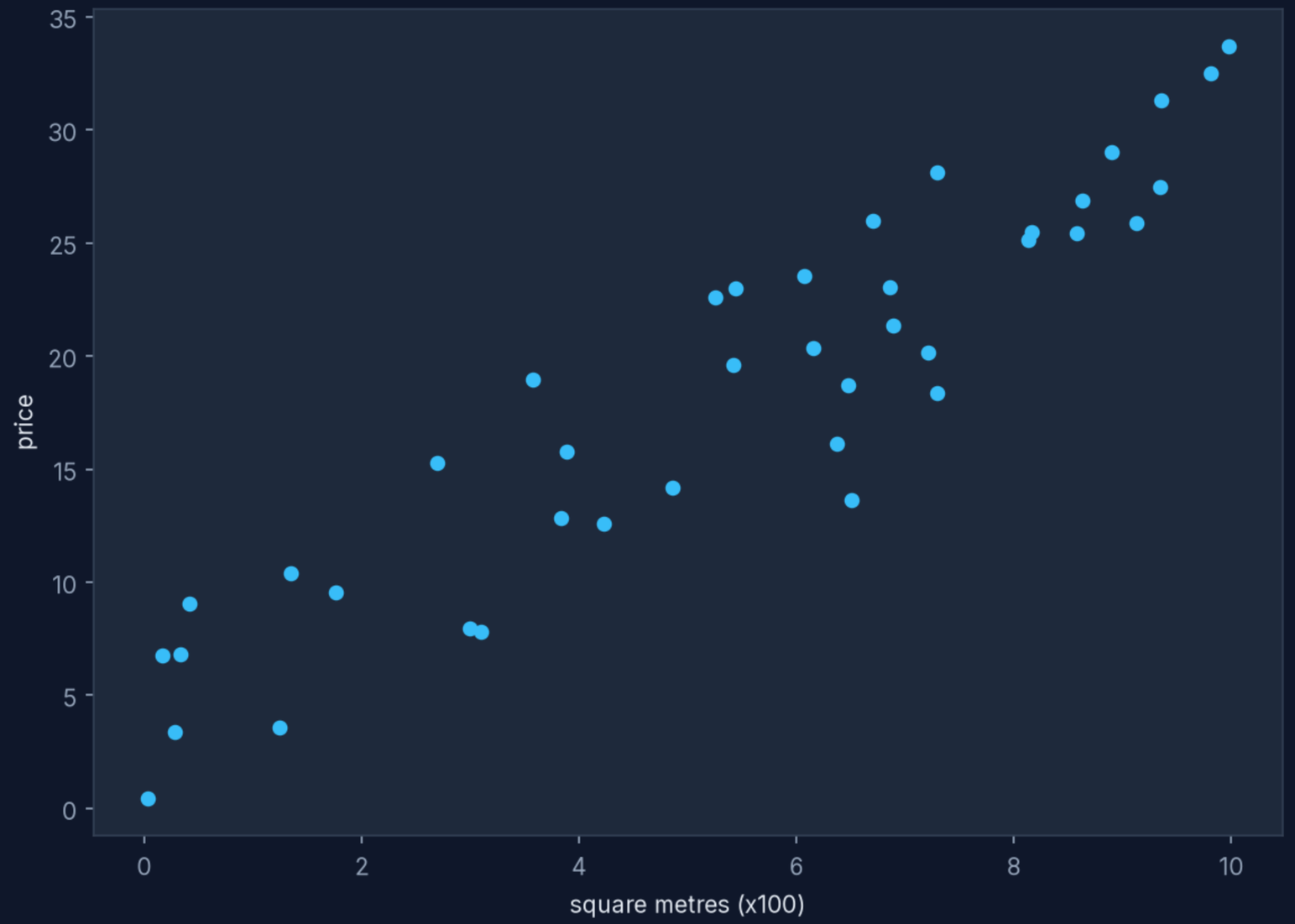
Linear Regression

Fitting the best straight line, two ways

The problem



Input: features X

Output: a number y

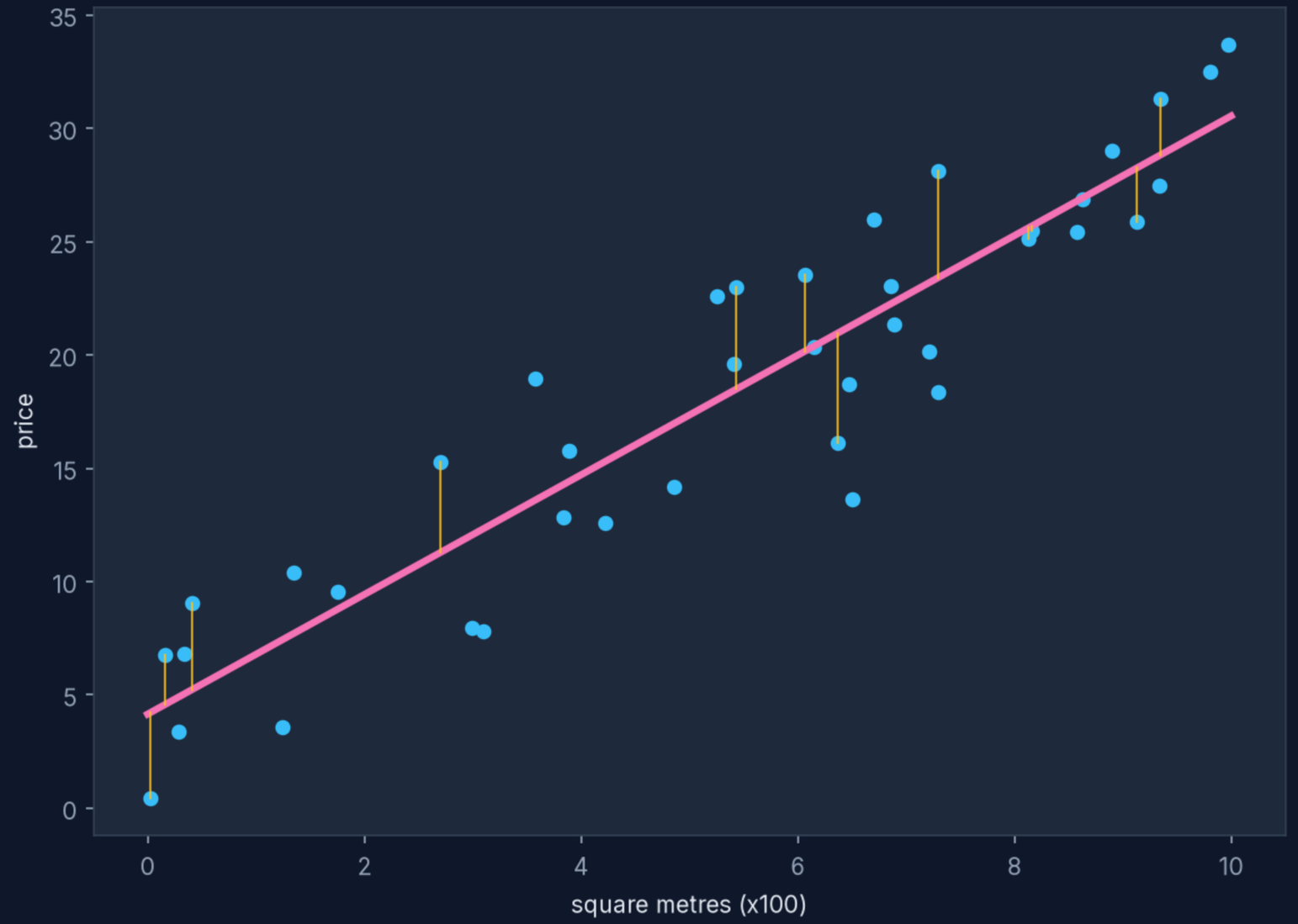
Goal: predict y for new, unseen X

Intuition: a weighted sum

$$\hat{y} = w_1 x_1 + \ \ldots \ + w_d x_d + b = Xw + b$$

- Each weight says how much a feature pushes the prediction
- The bias b is the baseline when every feature is zero

What does 'best' mean?



Residual = prediction – truth

Minimise mean squared error:

$$\text{MSE} = \frac{1}{n}\sum_{i}(\hat{y}_i - y_i)^2$$

Way 1: the normal equation

$$L(\theta) = \frac{1}{n}\|X_b\theta - y\|^2$$

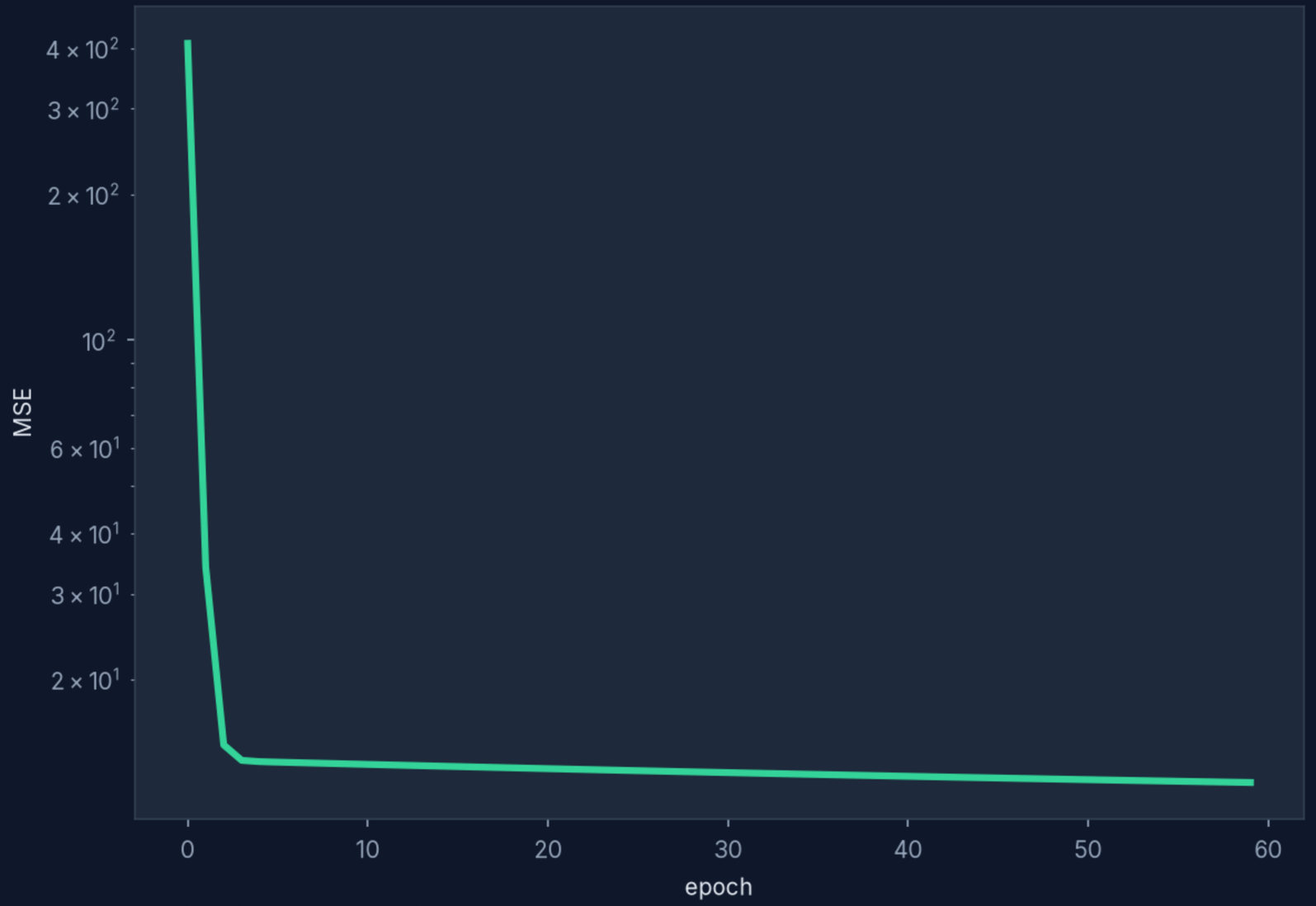
$$\theta = (X_b{}^\top X_b + \lambda I')^{-1} X_b{}^\top y$$

- Append a column of ones so the bias lives inside θ
- Set the gradient to zero → solve one linear system

Closed form in NumPy

```
Xb = np.hstack([X, np.ones((n, 1))])        # add bias column
reg = self.l2 * np.eye(d + 1)
reg[-1, -1] = 0                             # don't regularise bias
theta = np.linalg.solve(Xb.T @ Xb + reg, Xb.T @ y)
self.w, self.b = theta[:-1], theta[-1]
```

Way 2: gradient descent



$$\nabla_w = \frac{2}{n} X^\top (Xw + b - y)$$

$$w \leftarrow w - \eta \nabla_w$$

Repeat for many epochs

Gradient descent loop

```
self.w, self.b = np.zeros(d), 0.0
for _ in range(self.epochs):
    err = X @ self.w + self.b - y
    self.w -= self.lr * (2 * X.T @ err / n + 2 * self.l2 * self.w)
    self.b -= self.lr * 2 * err.mean()
```

Worked example

- Points: (1, 3), (2, 5), (3, 7)
- Means: $\bar{x} = 2$, $\bar{y} = 5$
- $w = \Sigma(x-\bar{x})(y-\bar{y}) \,/\, \Sigma(x-\bar{x})^2 = 4 \,/\, 2 = 2$
- $b = \bar{y} - w \cdot \bar{x} = 5 - 4 = 1 \;\rightarrow\; \hat{y} = 2x + 1$
- Residuals all zero: MSE = 0

Complexity & trade-offs

- Normal equation: $O(n \cdot d^2 + d^3)$ — exact, no tuning, slow for large d
- Gradient descent: $O(n \cdot d)$ per epoch — scales, needs learning rate & scaling
- Ridge ($\lambda > 0$) fixes ill-conditioned $X^T X$ and reduces overfitting
- Assumes a linear relationship; sensitive to outliers

Recap

- Prediction = weighted sum + bias
- Train by minimising mean squared error
- Closed form: solve $(X^TX + \lambda I)\theta = X^Ty$
- Gradient descent: step against the gradient